University of East Anglia
COMPUTING SCIENCES

Advanced Web Development

Exercise 4. Implementing registration and login UIs with Authentication by Google Provider via Auth.js and custom hook

Before writing any authentication code, it is critical to understand the architecture that keeps web applications secure. Security models typically rely on the AAA Framework:

- Authentication (Who are you?): Verifying the identity of a user or system. This is traditionally done using a combination of identifiers (username/email) and secrets (passwords), or via passwordless tokens and OAuth providers.
- Authorisation (What can you do?): Determining if an already authenticated user has permission to access a specific resource or perform a certain action (e.g., distinguishing an Admin from a Standard User).
- Accounting / Audit (What did you do?): Tracking user activity and logging actions. This helps identify security breaches and ensures accountability within the platform.

When a user successfully logs in, the server needs a way to remember them across stateless HTTP requests. JSON Web Tokens (JWT) is industry standard used for this purpose. It works by the server issues a signed cryptographic token to the client. The client sends this token with every request. The server decodes and verifies the signature without needing to query a database. This is highly scalable but makes token revocation harder. Another approach is the used of database sessions. This method relies on the server stores a session record in a database and sends a unique session ID cookie to the client. On every request, the server looks up that ID in the database. This allows instant session revocation but introduces extra database overhead.

Auth.js (formerly known as Next-Auth) is the evolution of Next.js authentication. It is a framework-agnostic, edge-compatible authentication library designed to abstract away complex security vulnerabilities (such as CSRF attacks, safe cookie management, and cryptographic token expiration).

There are three key architectural concepts of Auth.js.

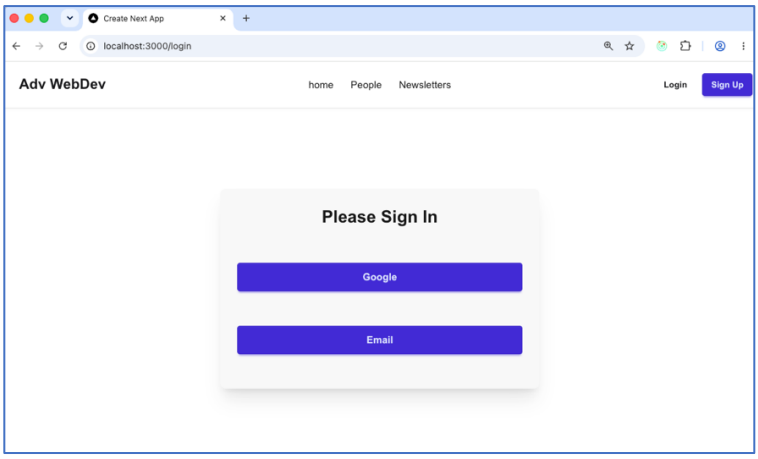
1. Providers: Define how users authenticate. Auth.js supports OAuth (Google, GitHub, etc.), WebAuthn (Passkeys), and Credentials (custom user databases).
2. Unified Configuration: Auth.js uses a centralised configuration object that easily exports route handlers, session fetching helpers (auth), and mutation controls (signIn, signOut).
3. Edge Compatibility: Core configuration files can run in middleware environments at the network edge, making authorization checks lightning fast.

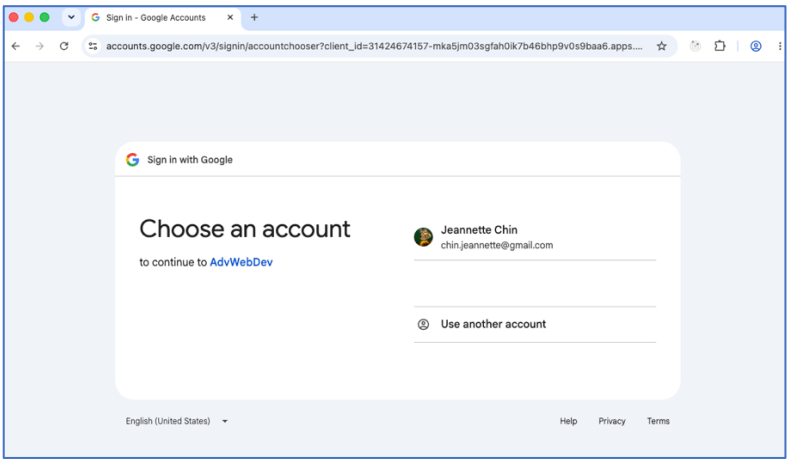
In this lab, we will implement 'login', 'logout' and 'register' functionalities using RTK state management tool. The implementation includes API programming using 'axios' library we learned

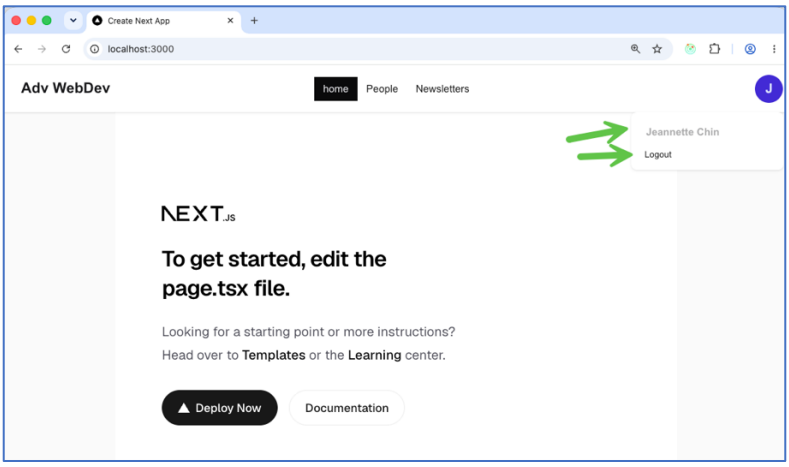
before, as well as examine client-side local storage. We will implement authentication using Next Auth.

To follow this lab, you must complete Excise 3.1 Next.js dynamic routes and RTK implementation first.

Example of intended outputs:





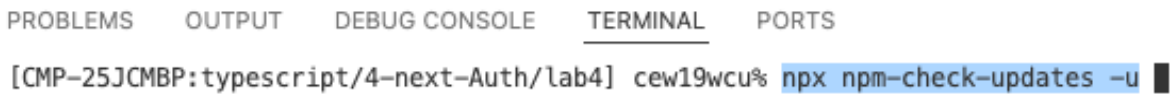


Learning Objectives:

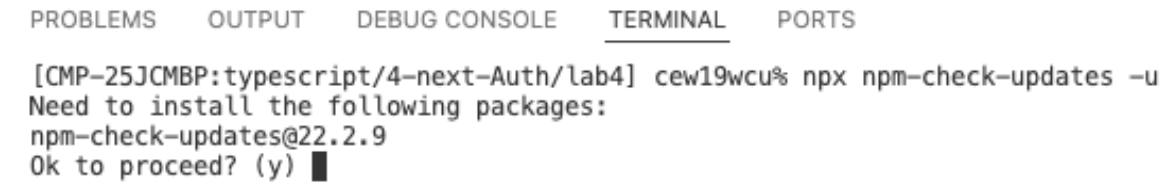
- Implement login and register UIs on Next.js
- Implement Google Provider using auth.js
- Implement custom hook - useContext
- Practicing React hooks

Tasks:

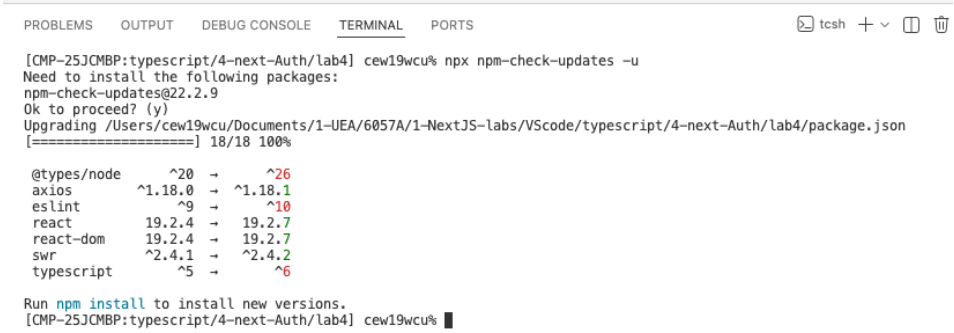
1. Create a new folder called 'lab4' and clone the code for lab3.1 inside this folder.
2. As we know it is important to keep our dependencies up to date. Once you have cloned the repo, open this new folder on VS Code, and on terminal window type '`npx npm-check-updates -u`'



Enter when prompt:



In my case, there are quite a few packages require update:



Type `npm install` to install them:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                    tcsh

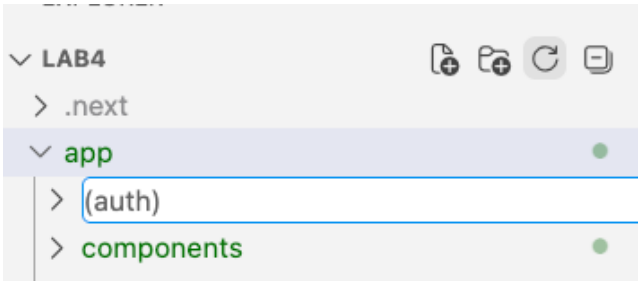
[CMP-25JCMBP:typescript/4-next-Auth/lab4] cew19wcu% npx npm-check-updates -u
Need to install the following packages:
npm-check-updates@22.2.9
Ok to proceed? (y)
Upgrading /Users/cew19wcu/Documents/1-UEA/6057A/1-NextJS-labs/VScode/typescript/4-next-Auth/lab4/package.json
[====================] 18/18 100%

 @types/node      ^20  →      ^26
 axios         ^1.18.0  →  ^1.18.1
 eslint            ^9  →      ^10
 react         19.2.4  →   19.2.7
 react-dom     19.2.4  →   19.2.7
 swr           ^2.4.1  →   ^2.4.2
 typescript        ^5  →       ^6

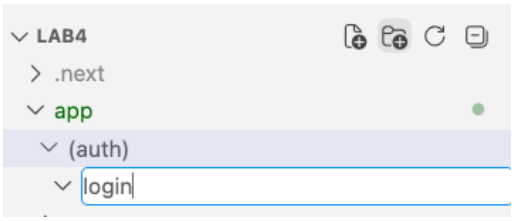
Run npm install to install new versions.
[CMP-25JCMBP:typescript/4-next-Auth/lab4] cew19wcu% npm install
```

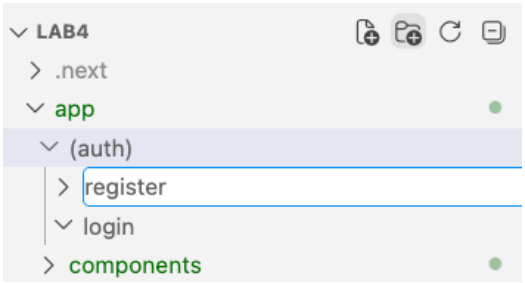
Now bring up your development server – `npm run dev`

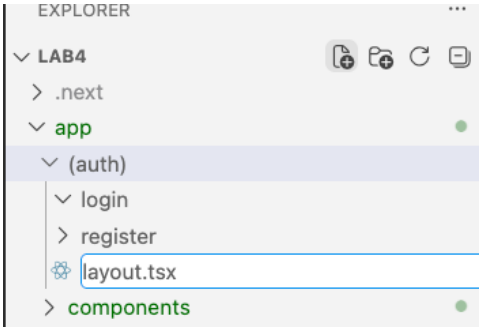
3. We will first code the Login and Register forms. Inside the app folder, create a new folder called (auth):



4. Inside this new folder, create 2 sub-folders : login and register, and a new file called `layout.tsx`, as follow:







5. Open the `layout.tsx` file, and code the following:

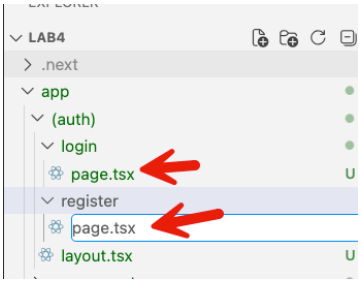
app > (auth) > layout.tsx > ...

```tsx
const AuthLayout = ({ children }: { children: React.ReactNode }) => {
  return (
    <div className="min-h-[calc(100vh-8rem)] flex items-center justify-center p-4">
      <div className="card w-full max-w-md bg-base-200 shadow-xl">
        <div className="card-body">{children}</div>
      </div>
    </div>
  );
};

export default AuthLayout;

```

6. Create a new file called `page.tsx` inside login and register folders:



7. Now open the `login.tsx` and code the LoginPage component (or type rafce for snippet plugin):

app > (auth) > login > page.tsx > ...

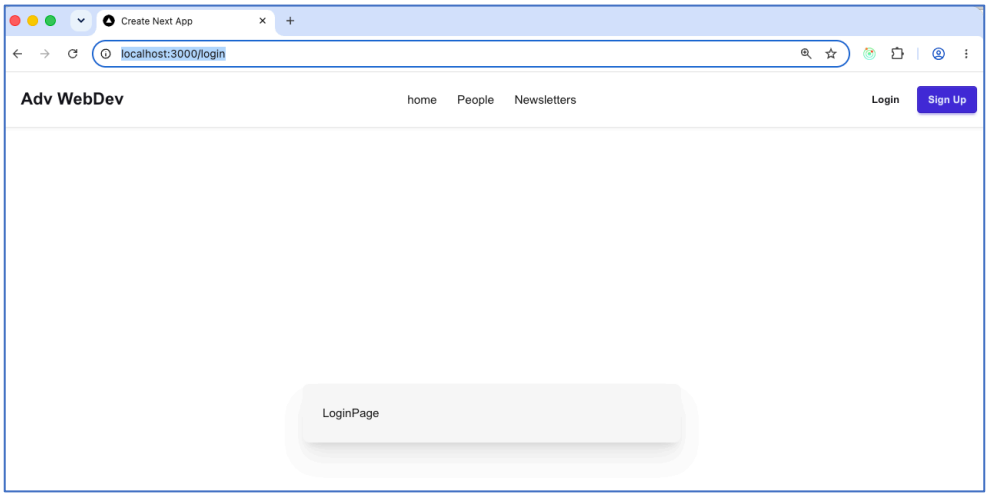
```tsx
import React from "react";

const LoginPage = () => {
  return <div>LoginPage</div>;
};

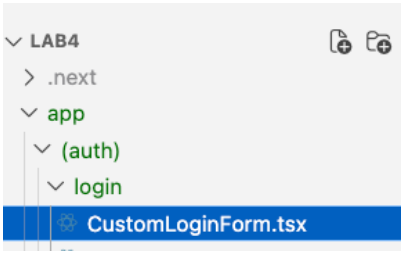
export default LoginPage;

```

8. Now navigate to http://localhost:3000/login or click the login button, you should see the output similar to below:



9. Inside the login folder, create a new file called "CustomLoginForm.tsx":



10. Open this `CustomLoginForm.tsx`, and code the Custom Login form –
    a. Make it a client component
    b. Code the login form with 2 input fields – email and password, and a sign in button. Below is my code:

app > (auth) > login > CustomLoginForm.tsx > ...

```tsx
"use client";
import Link from "next/link";
const CustomLoginForm = () => {
  return (
    <>
      <form className="mt-4">
        {" "}
        <div className="form-control">
          <label className="label">
            <span className="label-text">Email</span>
          </label>
          <input
            type="email"
            placeholder="you@example.com"
            className="input input-bordered w-full"
            required
          />
        </div>
        <div className="form-control mt-4">
          <label className="label">
            <span className="label-text">Password</span>
          </label>
          <input
            type="password"
            placeholder="••••••••"
            className="input input-bordered w-full"
            required
          />
        </div>
        <div className="form-control mt-6">
          <button type="submit" className="btn btn-primary w-full">
            Sign In
          </button>
        </div>
      </form>
      <div className="divider">OR</div>

      <p className="text-center">
        Don&apos;t have an account?{" "}
        <Link href="/register" className="link ■link-primary">
          Sign up
        </Link>
      </p>
    </>
  );
};

export default CustomLoginForm;
```

11. Inside the 'login' folder, create a new file called "`SiginButton.tsx`"



12. Open this file and code the following:

app > (auth) > login > SigninButton.tsx > ...

```tsx
const SigninButton = ({
  text,
  style,
  callback,
  formAction
}: {
  text: string;
  style: string;
  callback?: () => void;
  formAction?: any;
}) => {
  return (
    <div className="form-control mt-6">
      <button
        type="submit"
        formAction={formAction}
        className={style}
        onClick={callback}
      >
        {text}
      </button>
    </div>
  );
};

export default SigninButton;
```

Note: `callback` and `FormAction` are optional

13. Now navigate to login page (i.e. `page.tsx` inside the "login" folder) and code the following:

app > (auth) > login > page.tsx > ...

```tsx
"use client";

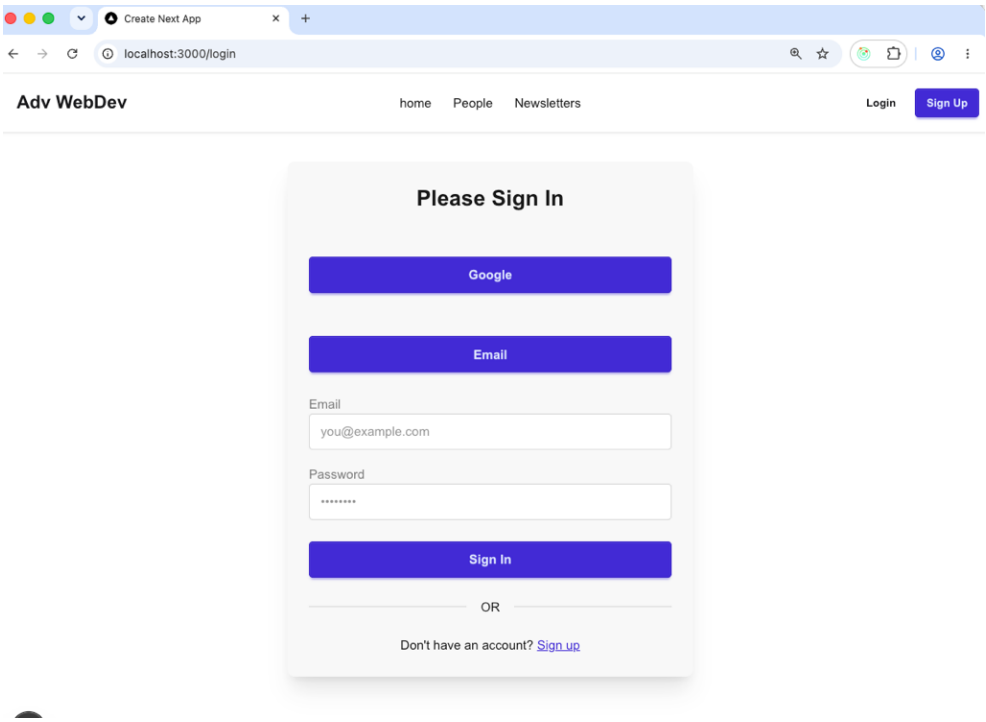
import CustomLoginForm from "./CustomLoginForm";

import SigninButton from "./SigninButton";

const LoginPage = () => {
  return (
    <>
      <h2 className="card-title text-2xl justify-center">Please Sign In</h2>
      <div className="mt-4">
        <SigninButton
          text="Google"
          style="btn btn-primary w-full"
          formAction={() => {}}
        />
      </div>
      <div className="mt-4">
        <SigninButton
          text="Email"
          style="btn btn-primary w-full"
          formAction={() => {}}
        />
      </div>
      <CustomLoginForm />
    </>
  );
};

export default LoginPage;
```

14. See the output on Chrome. Below is mine for your reference:



15. Here, we want to hide the custom login form by default, it only appears when the Email button is clicked. To achieve this behaviour, we use React `useState` hook. Navigate to the login page, and do the following 5 steps:
    a. Import `useState` hook.

```
5   import { useState } from "react";
```

    b. Create a state called `elogin`, initially set it to 'false'

```
8   const [elogin, setElogin] = useState(false);
```

    c. Code a handler called "`ShowEmailLogin`", to toggle the `elogin` state :

```
9   const ShowEmailLogin = () => {
10    setElogin(!elogin);
11  };
```

    d. Pass this handler to `callback` in the `SigninButton` for Email (note: change the `formAction` to `callback`)

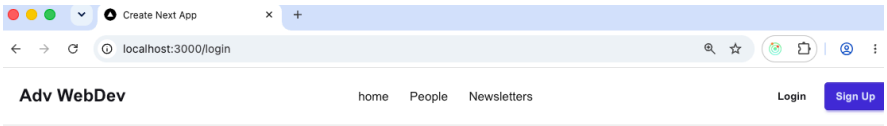
```
24      <SigninButton
25        text="Email"
26        style="btn btn-primary w-full"
27        callback={ShowEmailLogin}
28      />
```

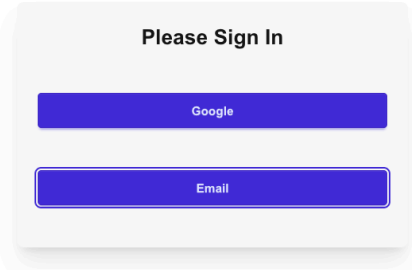
    e. In the return body, check this `elogin` state, if true, render the form, otherwise render nothing.

```
30      {elogin ? <CustomLoginForm /> : ""}
```

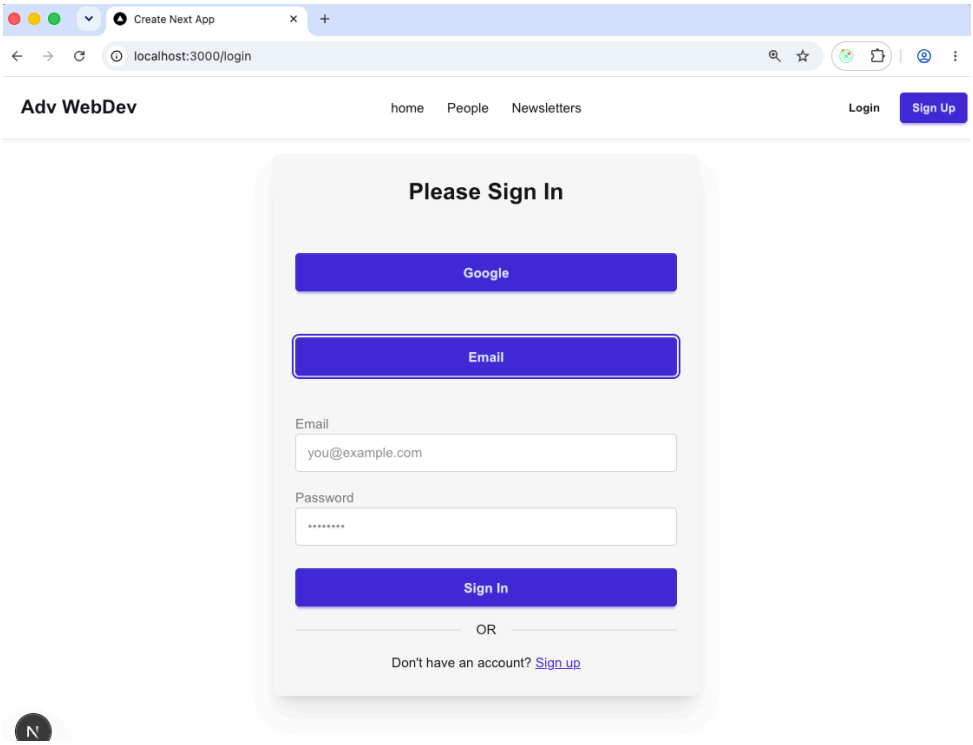
16. Now test it on Chrome:

    Login page:

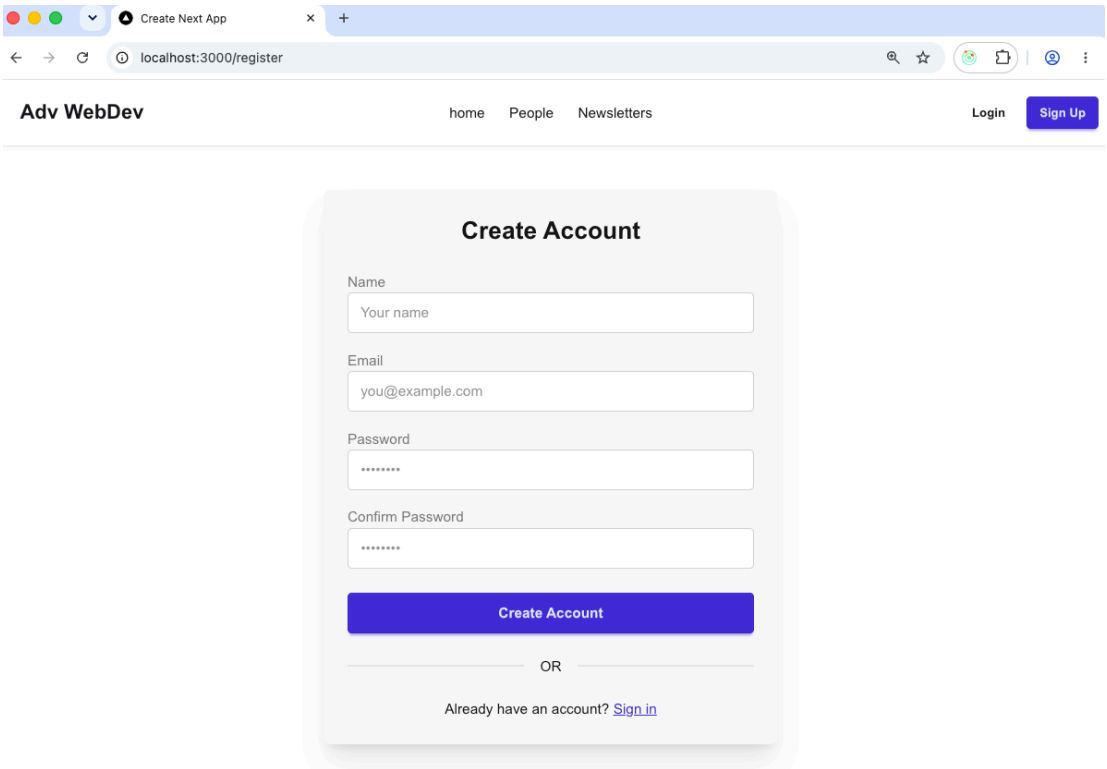




Click Email button:

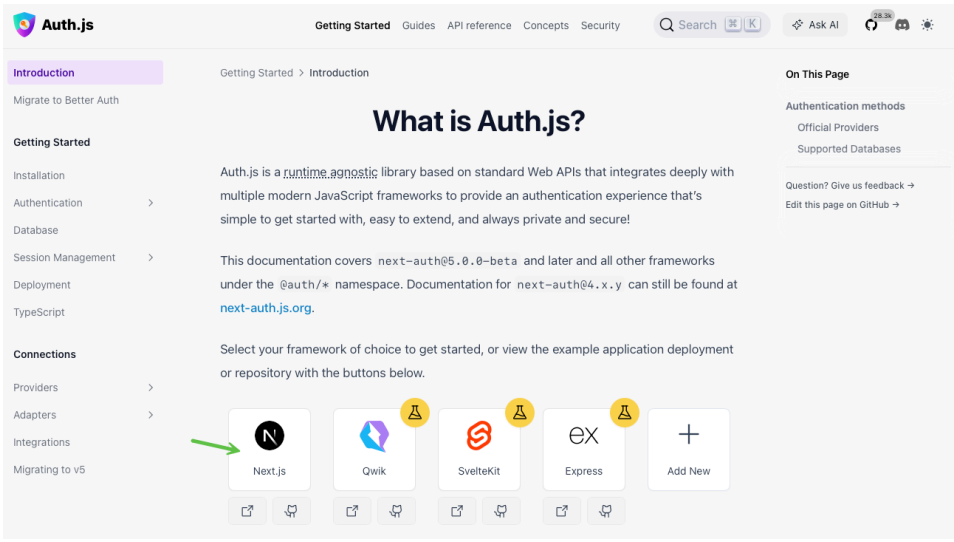


17. Once you have completed above, it's time to build the register form. Open the `page.tsx` inside the "register" folder and code the form. The form has 4 input fields and a button. Below is my output for your reference.

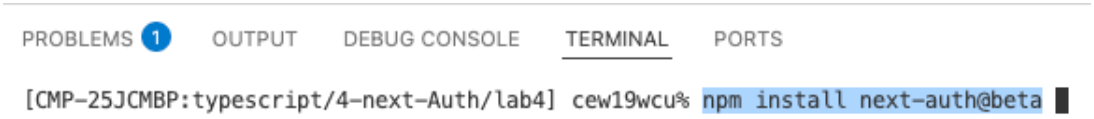


Test both links and make sure they work! We will implement the functionalities later.

18. When you have completed the UI construction above, we will move on to implement the Auth.js - https://authjs.dev . Go to : https://authjs.dev/getting-started and select Next.js



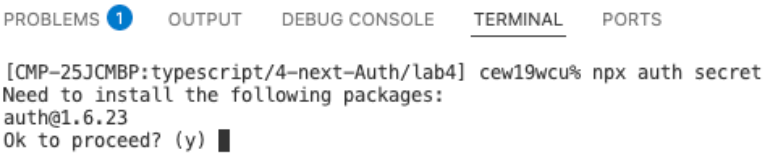
19. Stop your development server (Ctl C) , and install the library :
`npm install next-auth@beta`



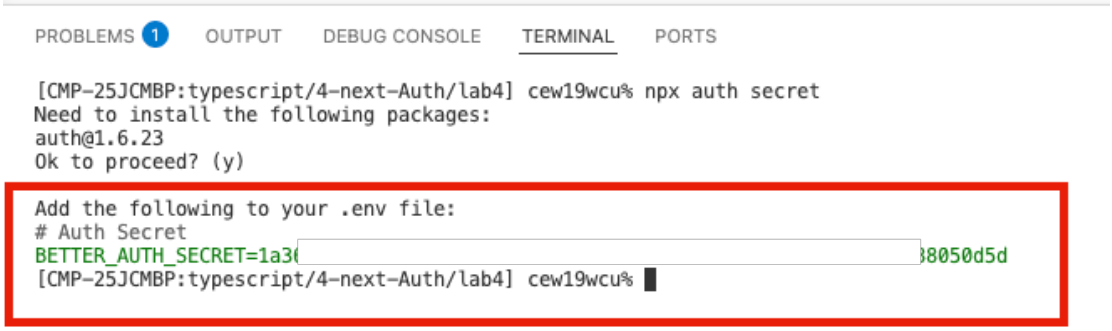
20. Then we will need to set up the environment: `npx auth secret`



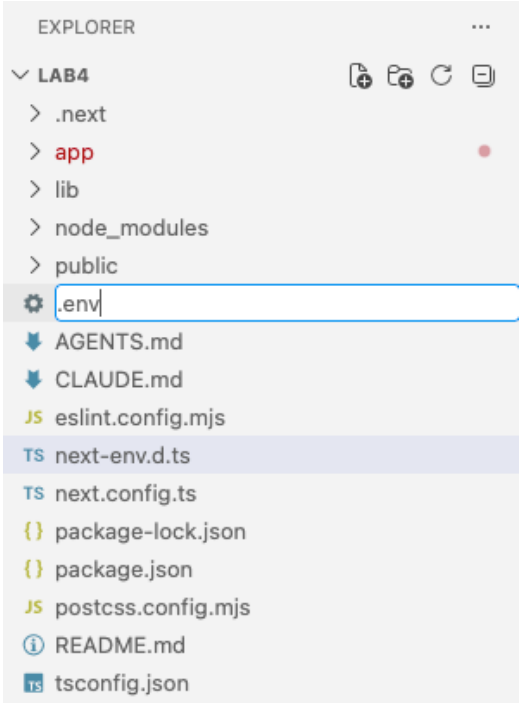
When prompt, enter



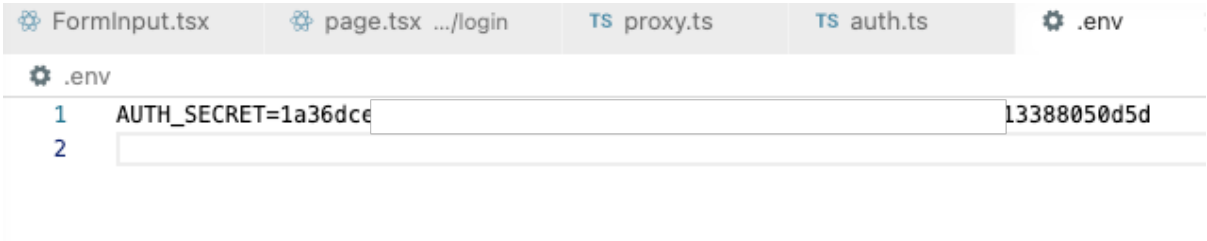
21. You will see a secret is being generated by Auth.js. Now copy this "BETTER_AUTH_SECRET" and paste it in a text editor, which we will use later:



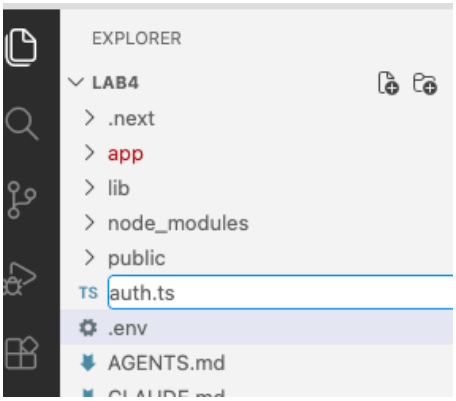
22. Create a `.env` file in your root folder:



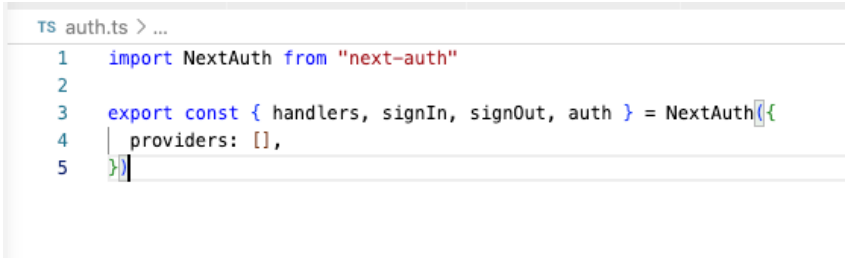
23. And paste the secret information you copied earlier over there, and make sure it is "AUTH_SECRET" NOT "BETTER_AUTH_SECRET", save the file.



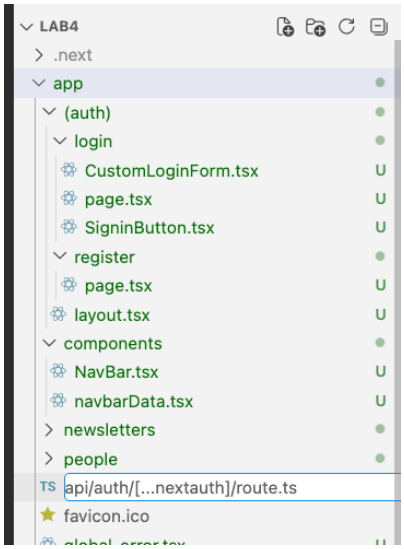
24. Next, we will create the `Auth.js` config file and object. This is where we can control the behaviour of the library and specify custom authentication logic, adapters, etc. It is recommended that an `auth.ts` file is created at the root of the app, so do the following:



25. The open the file, and copy the content from the website (https://authjs.dev/getting-started/installation?framework=Next.js) over to this file



26. Next add a route handler, inside the app folder, type `api/auth/[...nextauth]/route.ts`





The … dotation is referred to dynamic routes. You can find more information on Next.js site: https://nextjs.org/docs/pages/building-your-application/routing/dynamic-routes

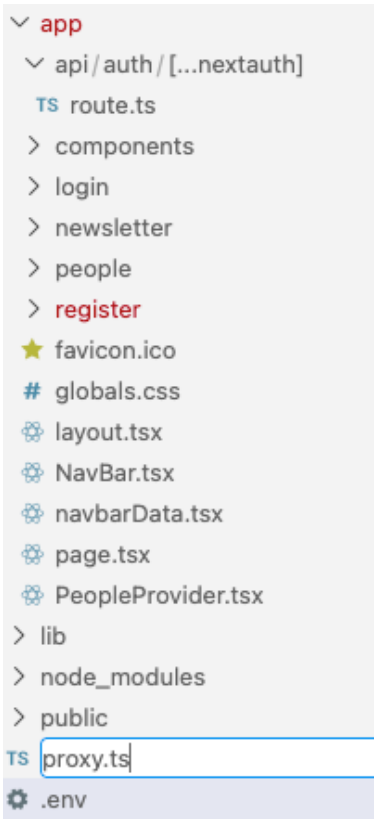
27. Open this `route.ts` file, and add the following lines:

app > api > auth > [...nextauth] > TS route.ts > ...

```ts
import { handlers } from "@/auth" // Referring to the auth.ts we just created
export const { GET, POST } = handlers

```

28. We will also need to configure a proxy to keep the session alive, this will update the session expiry every time it is called. So, create a `proxy.ts` in the **root** directory:



29. Open this `proxy.ts` file (old version is called `middleware.ts`) and add the following line:

TS proxy.ts

```ts
export { auth as proxy } from "@/auth"

```

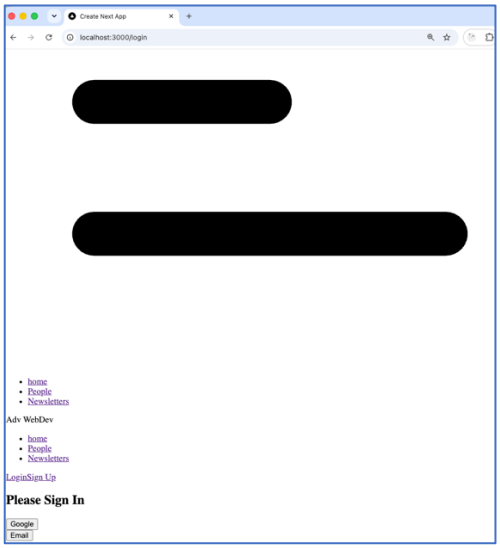
30. Now configure 'authorized' callback in our `auth.ts` file, so navigate to this file and add this block (line 5 to line 10):

```ts
TS auth.ts > ...
1   import NextAuth from "next-auth"
2
3   export const { handlers, signIn, signOut, auth } = NextAuth({
4     providers: [],
5     callbacks: {
6       authorized: async ({ auth }) => {
7         // Logged in users are authenticated, otherwise redirect to login page
8         return !!auth
9       },
10    },
11  })
12
```

31. Next, we will implement some logics in our `proxy.ts` , comment out the previous line, and add the following:

```ts
TS proxy.ts > ...
1   //export { auth as proxy } from "@/auth"
2
3   import { auth } from "@/auth"
4
5   export const proxy = auth((req) => {
6     if (!req.auth && req.nextUrl.pathname !== "/login") {
7       const newUrl = new URL("/login", req.nextUrl.origin)
8       return Response.redirect(newUrl)
9     }
10  })
11
12
```

32. Now restart the development server and test it out! As you will, whatever page you click, the app will take you to the login page, and the default CSS is gone:

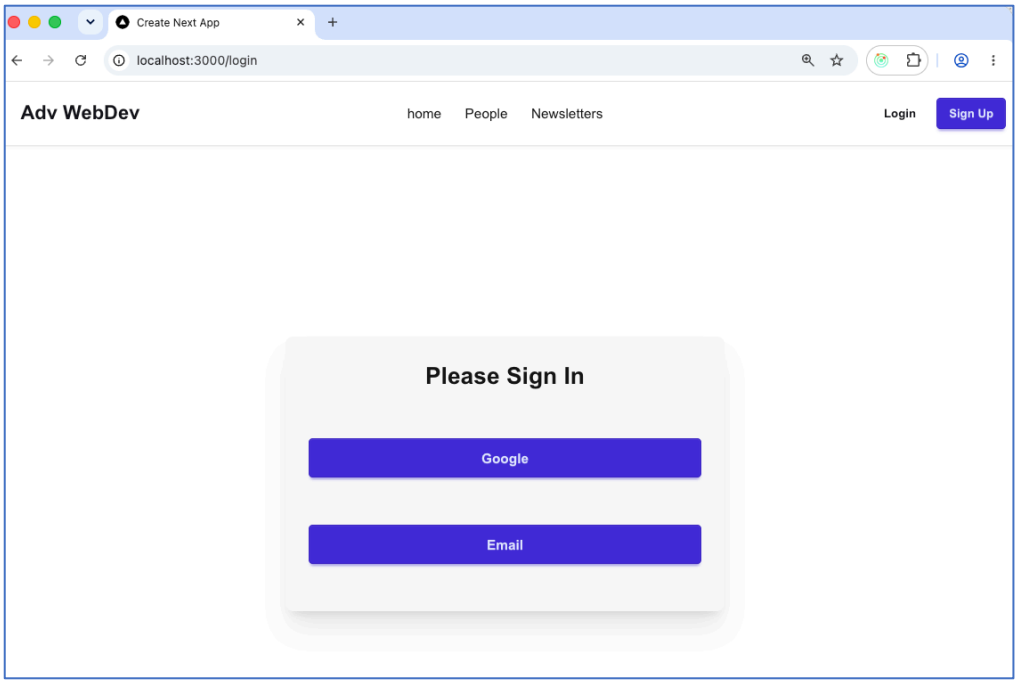


This behaviour is because the proxy does not know where the `css` files

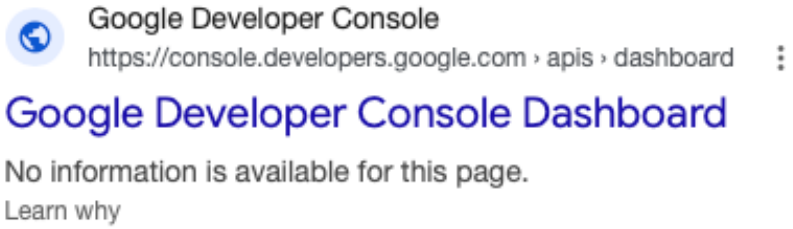
33. We will need to add one more line to the `proxy.ts` file, to match multiple routes or negate certain routes in order to protect all remaining routes. The following example avoids running the proxy on paths such as the favicon or static images.

```
12
13  export const config = {
14    matcher: ["/((?!api|_next/static|_next/image|favicon.ico).*)"],
15  }
16
```

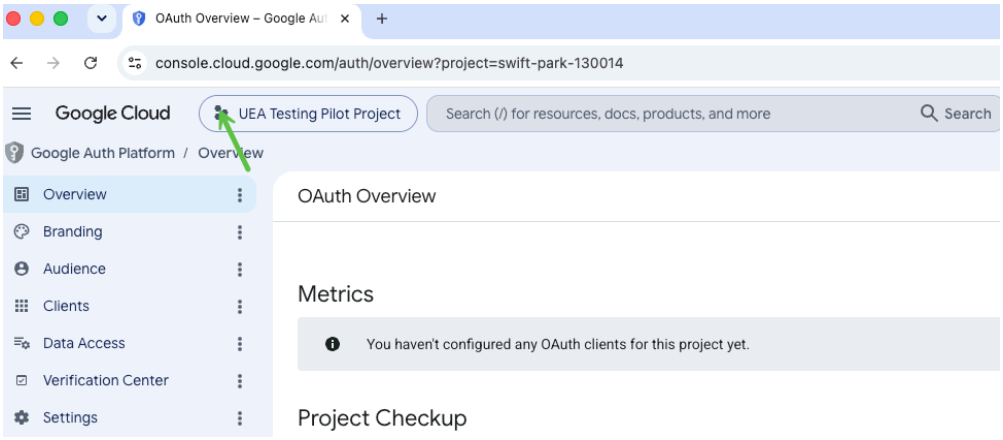
34. Now refresh the page, you will the login page. If you click other links, the app will always want you to login! Why? This is how we configure right now, see item 31 above.



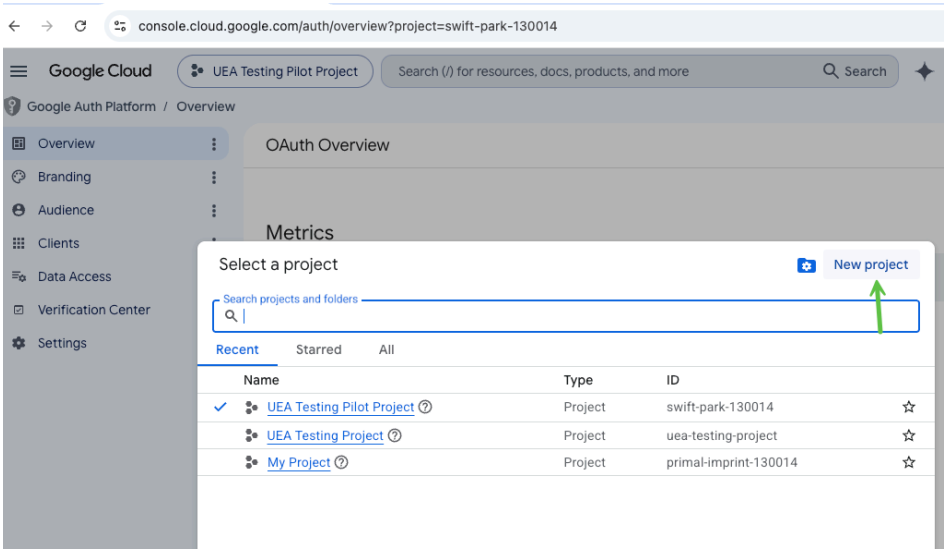
35. We will implement our provider for authentication next. Go to this page : https://authjs.dev/getting-started/authentication/oauth , you will see there are many OAuth providers. For this lab exercises, we will use Google and GitHub. For Google provider, you will need to setup an OAuth application on your Google developers dashboard. Go to Google developer dashboard and sign in



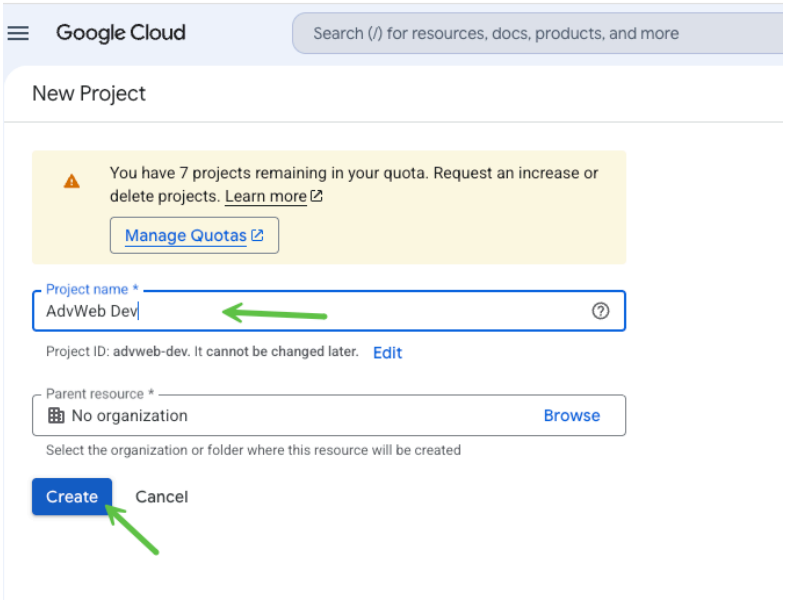
36. Go to Google Cloud https://console.cloud.google.com/ and sign in. Obviously, you will need to use your GMAIL account.



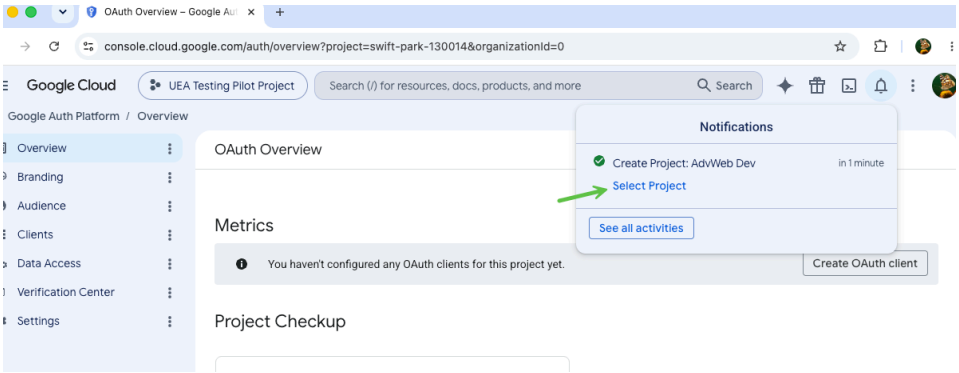
37. Create a new Project:



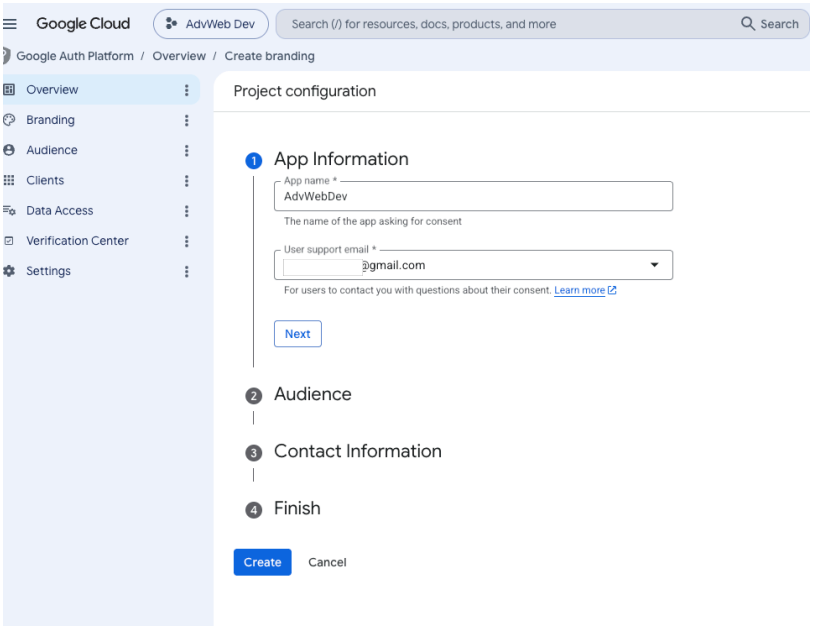
38. Give the project a name and click create:

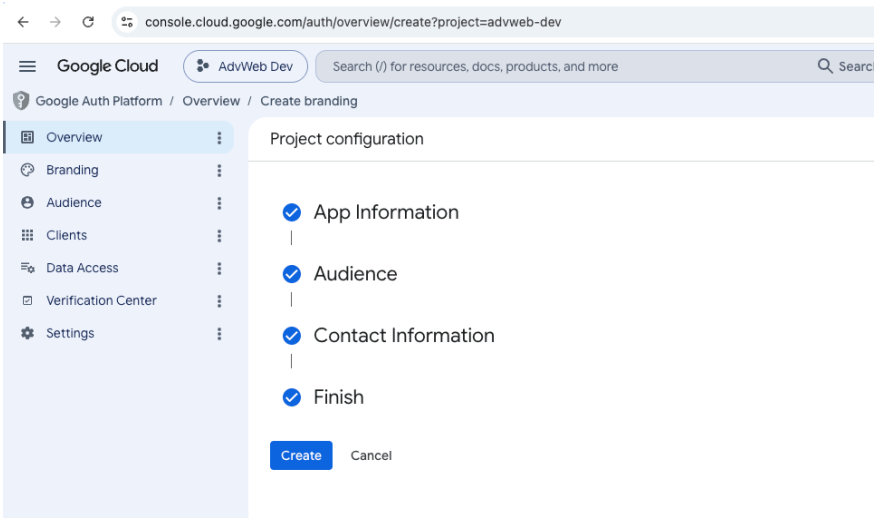


39. Now, navigate to this project:

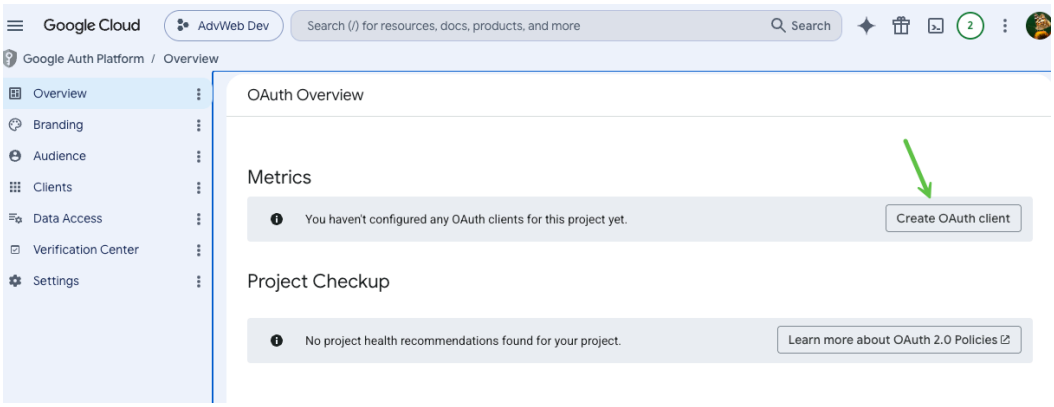


40. Follow the screen and fill in the details and click Next, and until Create:

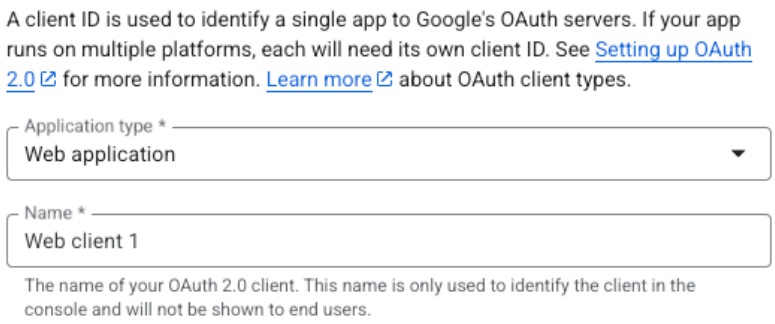




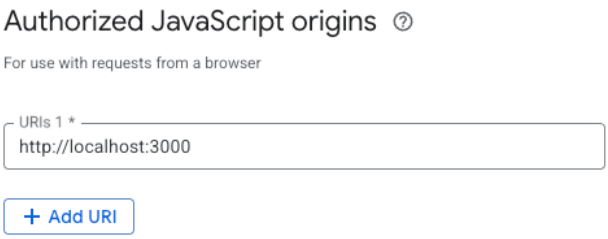
41. Once you are done, then configure the OAuth client:



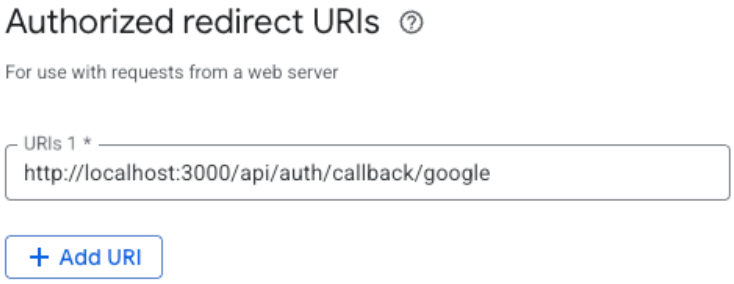
42. Select "Web Application" from the dropdown list, I will keep the default name:



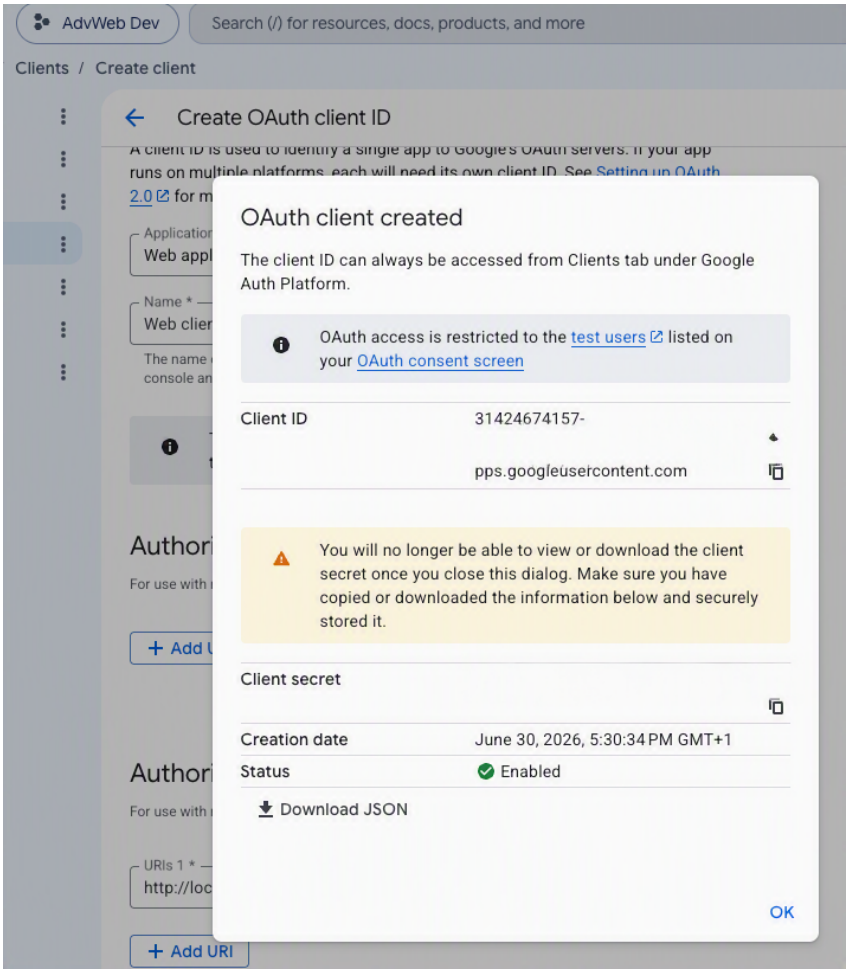
43. Here because we are in the development mode, but we want the provider to work with our current environment. We can configure the development URIs here as well.



Call back:



44. Click create, with a bit of luck you will see a screen similar to below:



It is important to copy the Client ID and secret now as it will disappear after you click OK. We will need this information later and click OK.

45. Navigate to your `.env` file, add the following lines:

AUTH_GOOGLE_ID={CLIENT_ID}
AUTH_GOOGLE_SECRET={CLIENT_SECRET}



Replace the placeholders with your Google client ID and secret as the values.

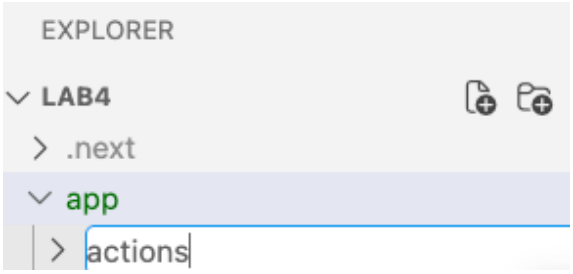
46. Next, we will add the Google Provider to our app. Navigate to `auth.ts` file and add the lines:

```ts
import NextAuth from "next-auth"
import Google from "next-auth/providers/google"

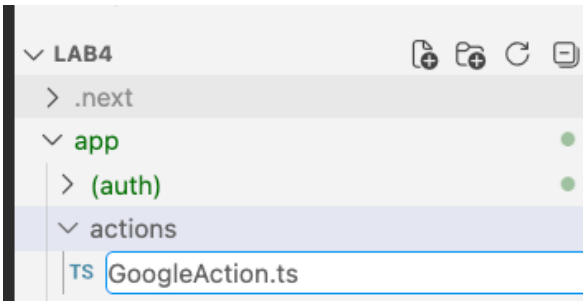
export const { handlers, signIn, signOut, auth } = NextAuth({
  providers: [Google],
  callbacks: {
    authorized: async ({ auth }) => {
      // Logged in users are authenticated, otherwise redirect to login page
      return !!auth
    },
  },

})
```

47. Inside the 'app' folder, create a new folder called 'actions'



48. Inside this new folder, create a file called `'GoogleAction.ts`"



49. Open this file and create and export Google sign in action:

app > actions > TS GoogleAction.ts > ...

```
1  "use server";
2
3  import { signIn } from "@/auth";
4
5  export async function handleGoogleLogin() {
6    await signIn("google", { redirectTo: "/" });
7  }
8
```

50. Navigate to `LoginPage` component (`page.tsx` in Login folder), and import this function:

app > (auth) > login > page.tsx > LoginPage

```
1  "use client";
2
3  import CustomLoginForm from "./CustomLoginForm";
4  import SigninButton from "./SigninButton";
5  import { useState } from "react";
6  import { handleGoogleLogin } from "@/app/actions/GoogleAction";
7
```

51. Then pass this action as `formAction` to the SubmitButton for Google (line 23)

```
20        <SigninButton
21          text="Google"
22          style="btn btn-primary w-full"
23          formAction={handleGoogleLogin}
24        />
```

A question: What happens if you attach the `FormAction` to `onClick` instead? Would it work? And why?

Answer: NO.

When you attach a server function to an onClick handler, Next.js treats it like a normal Asynchronous RPC (Remote Procedure Call).

- You click the button.
- The browser sends a background request (Fetch) to the server to run handleGoogleLogin.
- The server starts running signIn("google").

- Auth.js says, "Okay, I need to redirect this user to Google's login page." To do that on the server, it throws a special Next.js redirect signal.
- Because it was called via a background onClick fetch request, the browser receives a raw redirect response instead of actually changing the page URL. The JavaScript execution context gets confused, thinks the request failed or was called out of place, and throws that headers was called outside a request scope error.

However, when you pass a Server Action to a <form action={...}>, Next.js hooks into the browser's native form submission architecture.

- You click the button inside the form.
- Instead of a standard JavaScript click event, Next.js intercepts the form submission.
- It sends the request to the server wrapped in a specialized HTTP POST request built specifically for Server Actions.
- When Auth.js executes signIn("google") and issues the redirect command, Next.js looks at the incoming form request and says, "Ah, this is a form action submission. I am allowed to completely hijack the browser's main window navigation."

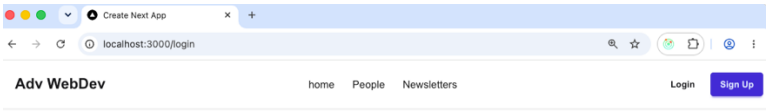
Next.js gracefully handles the redirect response and seamlessly pushes your entire browser tab over to Google's authentication page.

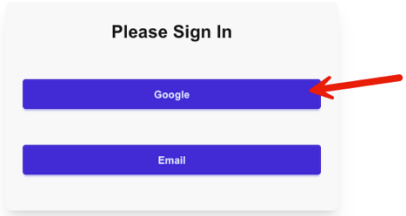
Also, we need to wrap the Google button inside `<form>` tag as `<div>` won't work!

```
19        <form className="mt-4">
20          <SigninButton
21            text="Google"
22            style="btn btn-primary w-full"
23            formAction={handleGoogleLogin}
24          />
25        </form>
```

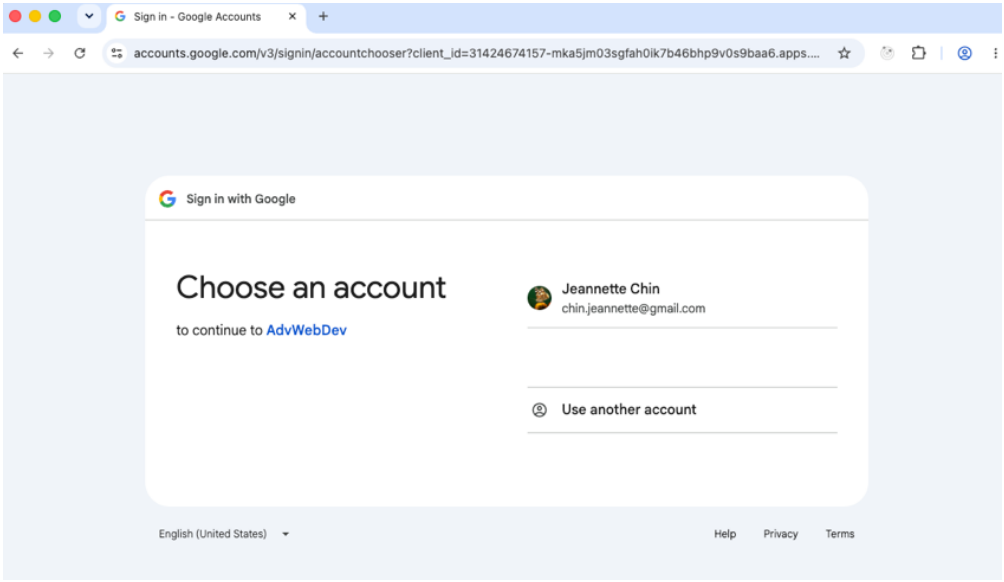
52. Now test the page to see if it works!

Step1: Click Google button

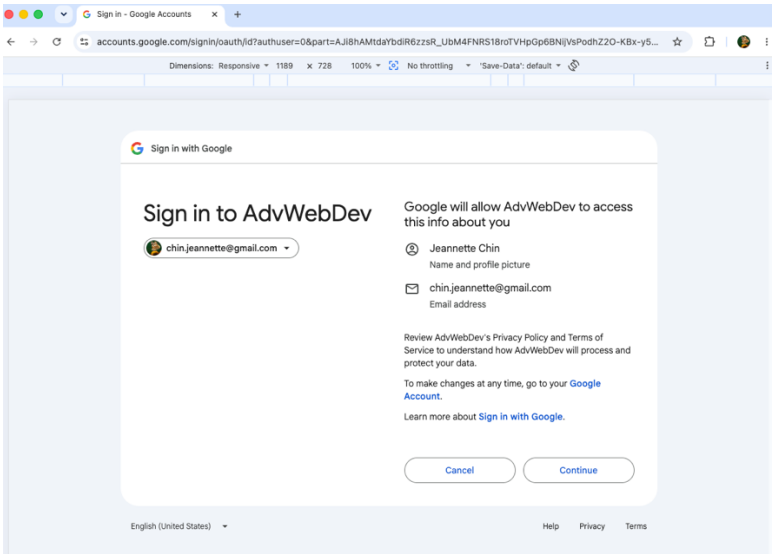




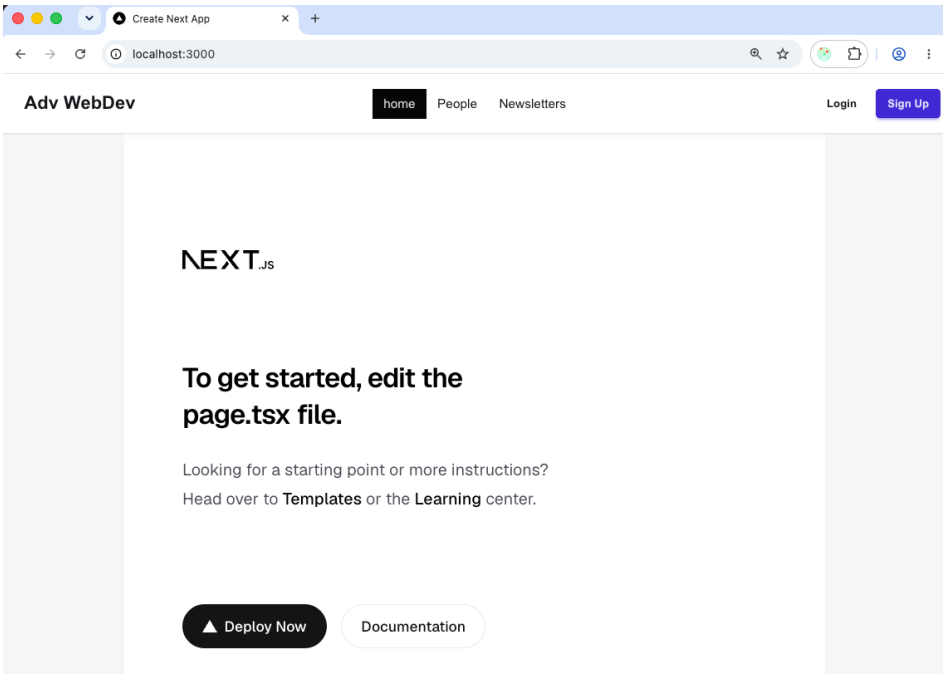
Step 2: - choose an account to login



Step 3: - click continue

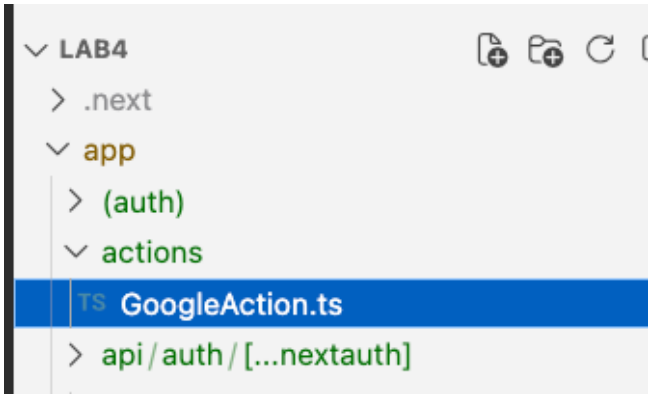


Step 4: you will be redirect to the homepage!

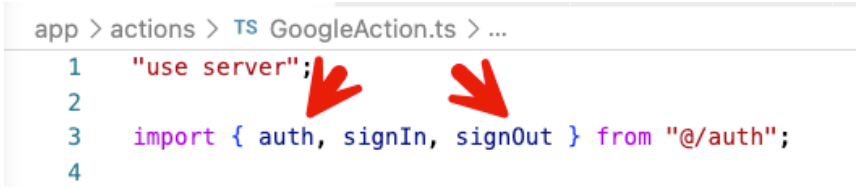


If you have achieved to this point well done! Notice that the login and Signup buttons are still showing on the Navbar even after the user is logged. We want to show the current login user's name and a logout button instead, which we will implement next.

53. First, navigate to the `GoogleAction.ts` file:



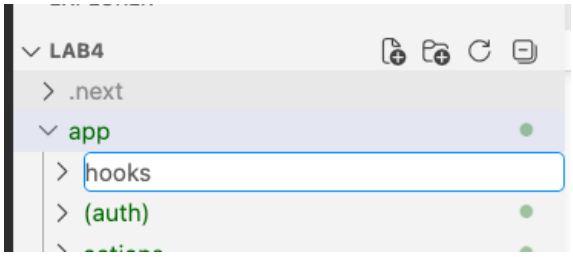
54. import `auth` and `signOut` from `auth.js`:



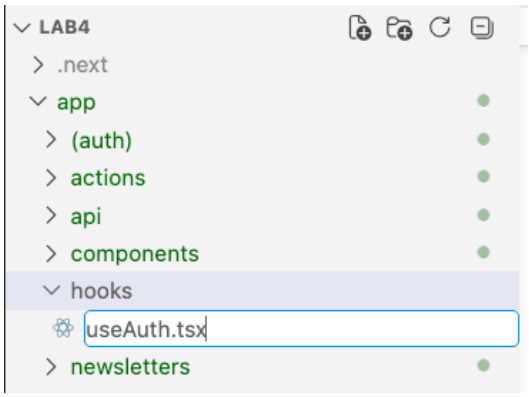
55. Then code two functions – one for returning the session from auth, and the other is for logout, as shown below:

```
9
10  export async function checkGoogleSession() {
11    const session = await auth();
12    return session;
13  }
14
15  export async function handleGoogleSignout() {
16    await signOut({ redirectTo: "/login" });
17  }
18
```

56. Inside the 'app' folder, create a new folder called 'hooks' :



57. We will now implement our custom hook. Inside this "hooks" folder, create a new file called `useAuth.tsx`:



58. Open this file, and do the following:
    a. Make this a client component
    b. Import React hooks and ReactNode:

```
3   import {
4     createContext,
5     useContext,
6     useState,
7     useEffect,
8     useCallback,
9     ReactNode
10  } from "react";
```

    c. Import `useRouter` hook from next/navigation:

```
12  import { useRouter } from "next/navigation";
```

    d. Import Google actions we just coded:

```
14  import {
15    checkGoogleSession,
16    handleGoogleSignout
17  } from "../actions/GoogleAction";
18
```

e. Now type our user. The User has id, name and email.

```
19  interface User {
20    id: string | null | undefined;
21    email: string | null | undefined;
22    name: string | null | undefined;
23  }
```

f. Type the Auth state. The app has 3 states : current user, isAuthenticated, and isLoading:

```
25  interface AuthState {
26    user: User;
27    isAuthenticated: boolean;
28    isLoading: boolean;
29  }
```

g. Type the Auth Context, which includes the AuthState above, and for now we want to include 2 functions – one for checking Google session, another one for logout:

```
45
46  interface AuthContextType extends AuthState {
47    checkGoogle: () => void;
48    logout: () => Promise<void>;
49  }
```

h. Initialise the context using `createContext` :

```
50
51  const AuthContext = createContext<AuthContextType | undefined>(undefined);
52
```

i. Finally, export a function name `AuthProvider` :

```
52
53  export function AuthProvider({ children }: { children: ReactNode }) {
54
55  }
```

j. Inside this `AuthProvider` function, code the following:

   i. Initialise the router:

```
54    const router = useRouter();
```

ii. Create a null user

```
56  const user = {
57    id: "",
58    email: "",
59    name: ""
60  };
```

iii. Create our auth state using React `useState` hook, and initialise it with null user, isAuthenticated false and isLoading true, as follow:

```
62  const [authState, setAuthState] = useState<AuthState>({
63    user: user,
64    isAuthenticated: false,
65    isLoading: true
66  });
```

iv. Code check Google session function as follow:

```
68  const checkGoogle = async () => {
69    const session = await checkGoogleSession();
70    if (session) {
71      const currentUser: User = {
72        id: session.user?.id,
73        name: session.user?.name,
74        email: session.user?.email
75      };
76      setAuthState({
77        user: currentUser,
78        isAuthenticated: true,
79        isLoading: false
80      });
81    }
82  };
```

v. We want to execute the above function once on mount:

```
113 // Check auth status on mount
114 useEffect(() => {
115   checkGoogle();
116 }, []);
117
```

vi. Code the logout function, as follow:

```
174   const logout = useCallback(async () => {
175     setAuthState({
176       user: user,
177       isAuthenticated: false,
178       isLoading: false
179     });
180
181     await handleGoogleSignout();
182   }, []);
183
```

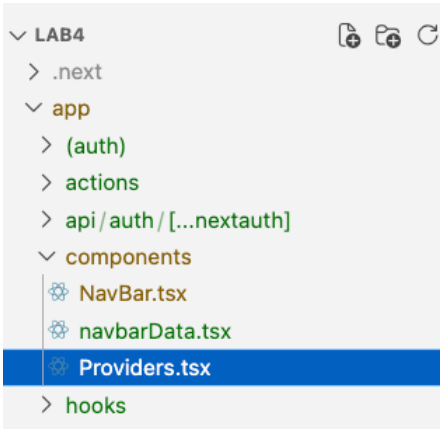
vii. Return the context provider as follow:

```
179   const value: AuthContextType = {
180     ...authState,
181     checkGoogle,
182     logout
183   };
184
185   return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
```

viii. Now we are ready export our custom hook, as follow:

```
188   /**
189    * useAuth hook - uses shared AuthContext
190    * All components using this hook share the same auth state
191    */
192   export function useAuth() {
193     const context = useContext(AuthContext);
194     if (context === undefined) {
195       throw new Error("useAuth must be used within an AuthProvider");
196     }
197     return context;
198   }
199
```

ix. To use this hook, first, create a new file named "`Providers.tsx`" inside components folder:



x. Open this file and code the following:

app > components > Providers.tsx > ...

```tsx
"use client";

import { AuthProvider } from "../hooks/useAuth";

export function Providers({ children }: { children: React.ReactNode }) {
  return <AuthProvider>{children}</AuthProvider>;
}

```

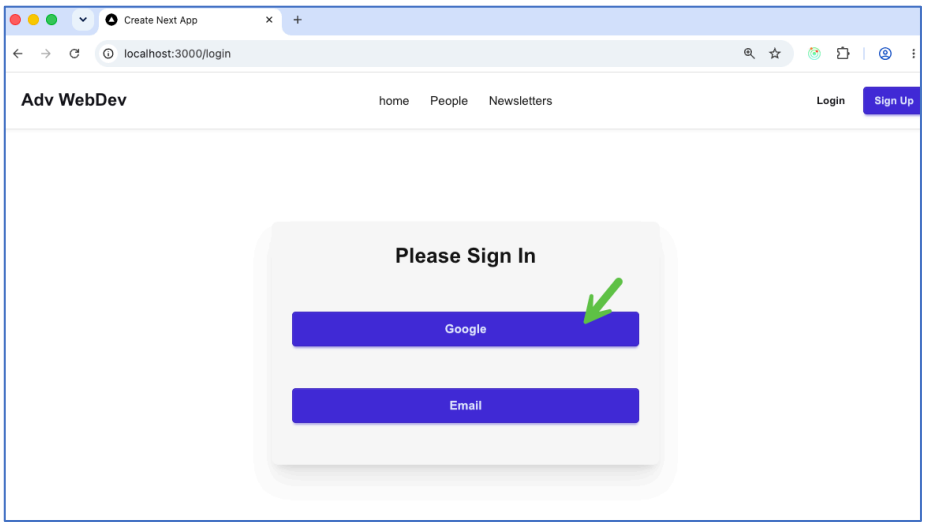
xi. Then navigate to `layout.tsx`, and import this provider:

```tsx
import { Providers } from "./components/Providers";
```

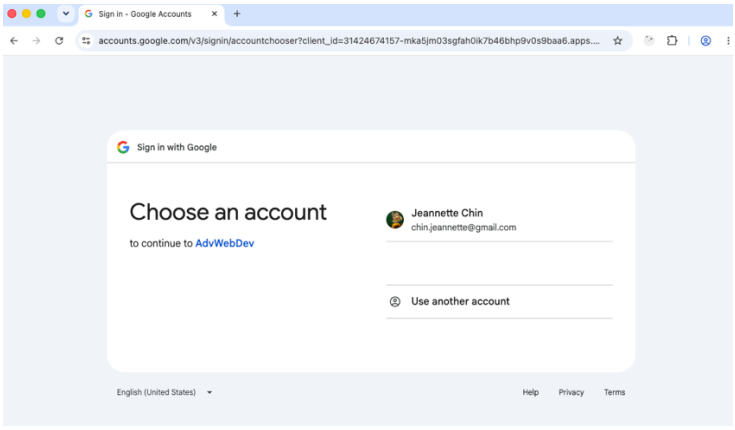
xii. Wrap the provider as follow:

```tsx
<body className="min-h-full flex flex-col">
  <Providers>
    <PeopleProvider>
      <Toaster position="bottom-right" />
      <NavBar />
      {children}
    </PeopleProvider>
  </Providers>
```

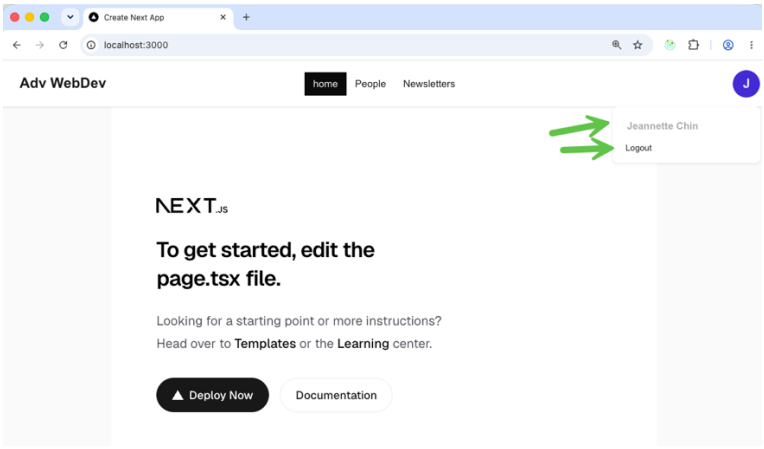
xiii. That's it! save the files and test it!
Step 1:



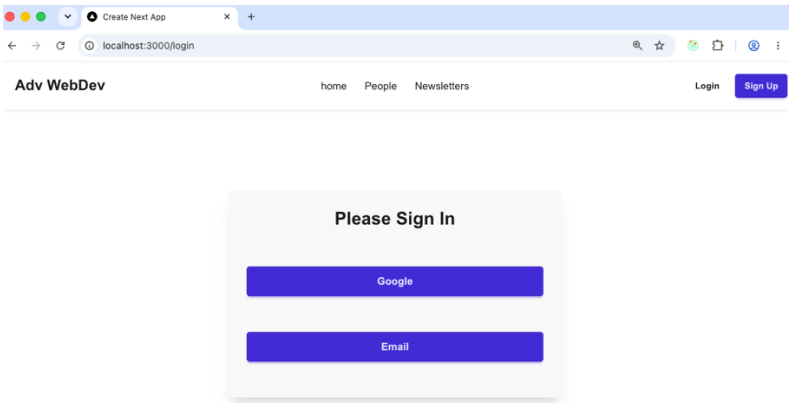
Step 2: select an account to login



Step 3: check user info, as well as able to navigate to different pages



Step 4: click Logout, you should be redirected to login page:



Once you have completed the above challenge, good job!

Your next task is to commit your code to CMP Stugit and name your repo as as "<your-username>lab4.0"

We will implement Postgres database next!